# 1.First Come First Serve (FCFS):

# C PROGRAM:

```
#include<stdio.h>
int main()
{
int n,i;
int bt[20],wt[20],tat[20];
float avg_wt=0,avg_tat=0;
printf("Enter Number of Processes: ");
scanf("%d",&n);
for(i=0;i<n;i++)
{
printf("Enter Burst Time for P%d: ",i+1);
scanf("%d",&bt[i]);
}
wt[0]=0;
for(i=1;i<n;i++)
wt[i]=wt[i-1]+bt[i-1];
for(i=0;i<n;i++){
tat[i]=wt[i]+bt[i];
avg_wt+=wt[i];
avg_tat+=tat[i];
```

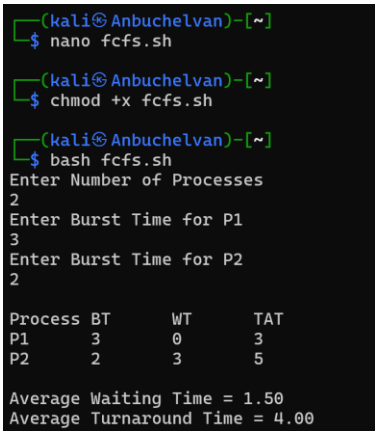
```
}
printf("\nProcess\tBT\tWT\tTAT\n");
for(i=0;i<n;i++)
printf("P%d\t%d\t%d\t%d\n",i+1,bt[i],wt[i],tat[i]);
printf("\nAverage Waiting Time = %.2f",avg_wt/n);
printf("\nAverage Turnaround Time = %.2f\n",avg_tat/n);
return 0;
}
```

## SHELL SCRIPTING:

```
#!/bin/bash
echo "Enter Number of Processes"
read n
for ((i=0;i<n;i++))
do
echo "Enter Burst Time for P$((i+1))"
read bt[$i]
done
wt[0]=0
for ((i=1;i<n;i++))
do
wt[$i]=$((wt[i-1]+bt[i-1]))
done
echo
```

```
echo -e "Process\tBT\tWT\tTAT"

total_wt=0

total_tat=0

for ((i=0;i<n;i++))

do

tat[$i]=$((wt[i]+bt[i]))

total_wt=$((total_wt+wt[i]))

total_tat=$((total_tat+tat[i]))

echo -e "P$((i+1))\t${bt[i]}\t${wt[i]}\t${tat[i]}"


done

avg_wt=$(awk "BEGIN {printf \"%.2f\", $total_wt/$n}")

avg_tat=$(awk "BEGIN {printf \"%.2f\", $total_tat/$n}")

echo

echo "Average Waiting Time = $avg_wt"

echo "Average Turnaround Time = $avg_tat"
```

# OUTPUT:



## 2. Shortest Job First (SJF)

```
#include<stdio.h>
int main()
{
int n,i,j,temp;
int bt[20],wt[20],tat[20];
float avg_wt=0, avg_tat=0;
printf("Enter Number of Processes: ");
scanf("%d",&n);
for(i=0;i<n;i++)
{
printf("Enter Burst Time for P%d: ",i+1);
scanf("%d",&bt[i]);
}
// Sorting burst times in ascending order
for(i=0;i<n-1;i++)
{
for(j=i+1;j<n;j++)
{
if(bt[i] > bt[j])
{
temp = bt[i];
bt[i] = bt[j];bt[j] = temp;
```

```
}
}
}
wt[0] = 0;
for(i=1;i<n;i++)
wt[i] = wt[i-1] + bt[i-1];
printf("\nProcess\tBT\tWT\tTAT\n");
for(i=0;i<n;i++)
{
tat[i] = wt[i] + bt[i];
avg_wt += wt[i];
avg_tat += tat[i];
printf("P%d\t%d\t%d\t%d\n",i+1,bt[i],wt[i],tat[i]);
}
avg_wt = avg_wt / n;
avg_tat = avg_tat / n;
printf("\nAverage Waiting Time = %.2f", avg_wt);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat);
return 0;
}
```

## SHELL SCRIPTING:

```
#!/bin/bash
echo "Enter Number of Processes"
read n
for ((i=0;i<n;i++))
do
echo "Enter Burst Time for P$((i+1))"
read bt[$i]
done
# Sorting burst times (Ascending Order)
for ((i=0;i<n-1;i++))
do
for ((j=i+1;j<n;j++))
do
if [ ${bt[i]} -gt ${bt[j]} ]
then
temp=${bt[i]}
bt[$i]=${bt[j]}
bt[$j]=$temp
fi
done
done
wt[0]=0
```

```
echo
echo -e "Process\tBT\tWT\tTAT"
total_wt=0
total_tat=0
for ((i=0;i<n;i++))
do
if [ $i -ne 0 ]
then
wt[$i]=$((wt[i-1]+bt[i-1]))
fi
tat[$i]=$((wt[i]+bt[i]))
total_wt=$((total_wt+wt[i]))
total_tat=$((total_tat+tat[i]))
echo -e "P$((i+1))\t${bt[i]}\t${wt[i]}\t${tat[i]}"
done
avg_wt=$(awk "BEGIN {printf \"%.2f\", $total_wt/$n}")
avg_tat=$(awk "BEGIN {printf \"%.2f\", $total_tat/$n}")
echo
echo "Average Waiting Time = $avg_wt"
echo "Average Turnaround Time = $avg_tat"
```

OUTPUT:

```
┌──(kali㉿Anbuchelvan)-[~]
└─$ nano sjs.sh

┌──(kali㉿Anbuchelvan)-[~]
└─$ chmod +x sjs.sh

┌──(kali㉿Anbuchelvan)-[~]
└─$ bash sjs.sh
Enter Number of Processes
2
Enter Burst Time for P1
1
Enter Burst Time for P2
2

Process BT       WT       TAT
P1      1        0        1
P2      2        1        3

Average Waiting Time = 0.50
Average Turnaround Time = 2.00
```

## 3. Priority Scheduling:

## C PROGRAM:

```
#include<stdio.h>

int main() {

int n,i,j,temp;

int bt[20],pr[20],wt[20],tat[20];

float avg_wt=0, avg_tat=0;

printf("Enter Number of Processes: ");

scanf("%d",&n);
```

```
for(i=0;i<n;i++)
{
printf("

\nEnter Burst Time for P%d: ",i+1);

scanf("%d",&bt[i]);
printf("Enter Priority for P%d: ",i+1);
scanf("%d",&pr[i]);
}
/* Sort according to priority */
for(i=0;i<n
-1;i++)
{
for(j=i+1;j<n;j++)
{
if(pr[i] > pr[j])
{
temp = pr[i];
pr[i] = pr[j];
pr[j] = temp;
temp = bt[i];
bt[i] = bt[j];
```

```
bt[j] = temp;
}
}
}
wt[0] = 0;
for(i=1;i<n;i++)
wt[i] = wt[i
-1] + bt[i
-1];
printf("\nProcess\tPriority\tBT\tWT\tTAT\n");
for(i=0;i<n;i++)
{
tat[i] = wt[i] + bt[i];
avg_wt += wt[i];
avg_tat += tat[i];
printf("P%d\t%d\t\t%d\t%d\t%d\n",
i+1, pr[i], bt[i], wt[i], tat[i]);
}
avg_wt /= n;
avg_tat /= n;
printf("\nAverage Waiting Time = %.2f", avg_wt);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat);
return0;}
```

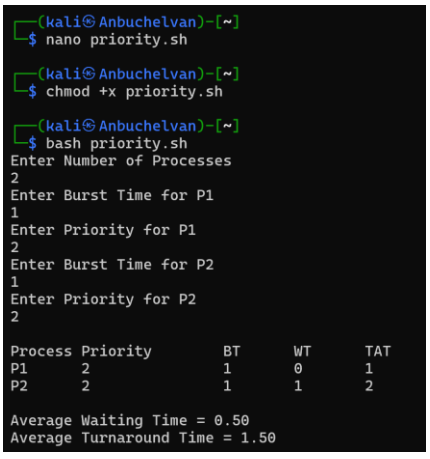
SHELL SCRIPTING:

```
#!/bin/bash
echo "Enter Number of Processes"
read n
for ((i=0;i<n;i++))
do
echo "Enter Burst Time for P$((i+1))"
read bt[$i]
echo "Enter Priority for P$((i+1))"
read pr[$i]
done
# Sort according to Priority
for ((i=0;i<n-1;i++))
do
for ((j=i+1;j<n;j++))
do
if [ ${pr[i]} -gt ${pr[j]} ]
then
```

```
temp=${pr[i]}
pr[$i]=${pr[j]}
pr[$j]=$temp
temp=${bt[i]}
bt[$i]=${bt[j]}
bt[$j]=$temp
fi
done
done
wt[0]=0
echo
echo -e "Process\tPriority\tBT\tWT\tTAT"
total_wt=0
total_tat=0
for ((i=0;i<n;i++))
do
if [ $i -ne 0 ]
then
```

```
wt[$i]=$((wt[i-1]+bt[i-1]))
fi
tat[$i]=$((wt[i]+bt[i]))
total_wt=$((total_wt+wt[i]))
total_tat=$((total_tat+tat[i]))
echo -e "P$((i+1))\t${pr[i]}\t\t${bt[i]}\t${wt[i]}\t${tat[i]}"
done
avg_wt=$(awk "BEGIN {printf \"%.2f\", $total_wt/$n}")
avg_tat=$(awk "BEGIN {printf \"%.2f\", $total_tat/$n}")
echo
echo "Average Waiting Time = $avg_wt"
echo "Average Turnaround Time = $avg_tat"
```

OUTPUT:



# 4. Round Robin Scheduling:

# C PROGRAM:

```
#include<stdio.h>
int main()
{
int n, tq, i;
int bt[20], rem_bt[20];
int wt[20] = {0}, tat[20];
int time = 0, done;
float avg_wt = 0, avg_tat = 0;
printf("Enter Number of Processes: ");
scanf("%d", &n);
for(i = 0; i < n; i++)
{
printf("Enter Burst Time for P%d: ", i + 1);
scanf("%d", &bt[i]);
rem_bt[i] = bt[i];
}
printf("Enter Time Quantum: ");
scanf("%d", &tq);
do
{
```

```
done = 1;
for(i = 0; i < n; i++)
{
if(rem_bt[i] > 0)
{
done = 0;
if(rem_bt[i] > tq)
{
time += tq;
rem_bt[i]-= tq;
}
else
{
time += rem_bt[i];
wt[i] = time
- bt[i];
rem_bt[i] = 0;
}
}
}
} while(!done);
printf("\nProcess\tBT\tWT\tTAT\n");
for(i = 0; i < n; i++)
```

```
{
tat[i] = bt[i] + wt[i];
avg_wt += wt[i];
avg_tat += tat[i];
printf("P%d\t%d\t%d\t%d\n",
i + 1, bt[i], wt[i], tat[i]);
}
avg_wt /= n;
avg_tat /= n;
printf("\nAverage Waiting Time = %.2f", avg_wt);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat);
return 0;
}
```

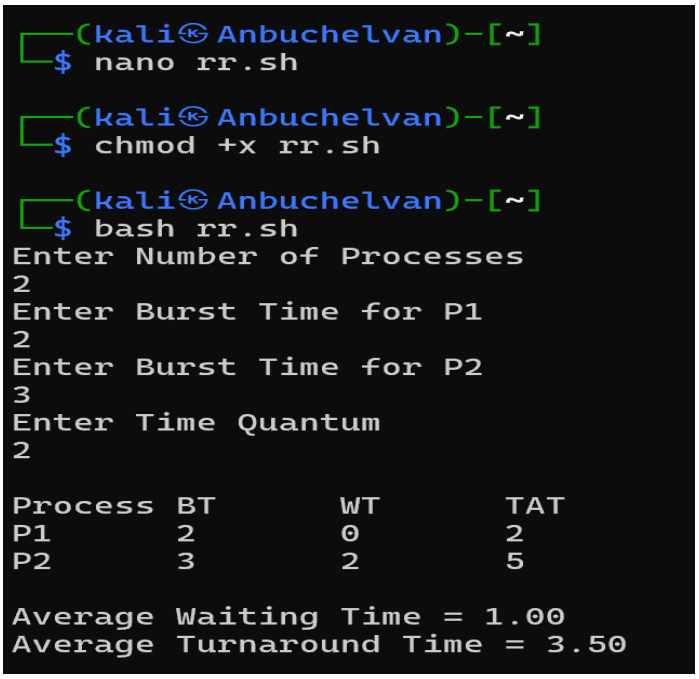
## SHELL SCRIPTING:

```
#!/bin/bash
echo "Enter Number of Processes"
read n
for ((i=0;i<n;i++))
do
echo "Enter Burst Time for P$((i+1))"
read bt[$i]
rem_bt[$i]=${bt[$i]}
done
```

```
echo "Enter Time Quantum"
read tq
time=0
while true
do
done=1
for ((i=0;i<n;i++))
do
if [ ${rem_bt[i]} -gt 0 ]
then
done=0
if [ ${rem_bt[i]} -gt $tq ]
then
time=$((time+tq))
rem_bt[$i]=$((rem_bt[i]-tq))
else
time=$((time+rem_bt[i]))
wt[$i]=$((time-bt[i]))
rem_bt[$i]=0
fi
fi
done
[ $done -eq 1 ] && break
```

```
done
echo
echo -e "Process\tBT\tWT\tTAT"
total_wt=0
total_tat=0
for ((i=0;i<n;i++))
do
tat[$i]=$((bt[i]+wt[i]))
total_wt=$((total_wt+wt[i]))
total_tat=$((total_tat+tat[i]))
echo -e "P$((i+1))\t${bt[i]}\t${wt[i]}\t${tat[i]}"
done
avg_wt=$(awk "BEGIN {printf \"%.2f\", $total_wt/$n}")
avg_tat=$(awk "BEGIN {printf \"%.2f\", $total_tat/$n}")
echo
echo "Average Waiting Time = $avg_wt"
echo "Average Turnaround Time = $avg_tat"
```

OUTPUT:


┌──(kali㉿Anbuchelvan)-[~]
└─$ nano rr.sh
┌──(kali㉿Anbuchelvan)-[~]
└─$ chmod +x rr.sh
┌──(kali㉿Anbuchelvan)-[~]
└─$ bash rr.sh
Enter Number of Processes
2
Enter Burst Time for P1
2
Enter Burst Time for P2
3
Enter Time Quantum
2
Process BT WT TAT
P1 2 0 2
P2 3 2 5
Average Waiting Time = 1.00
Average Turnaround Time = 3.50